

EE708: Fundamentals of Data Science and Machine Intelligence

Rajesh M Hegde
Dept. of EE IIT Kanpur



Module 4B Decision Trees

Machine Learning vs. Statistics in Predictive Modeling

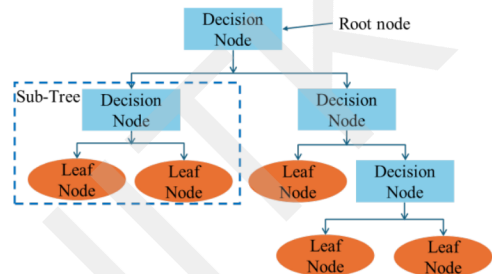
- Focus of Machine Learning:
 - Emphasis on algorithms that scale with large data.
 - Aims to optimize predictive models.
- Focus of Statistics:
 - Concentrates on probabilistic theory and model structure.
- Key Techniques:
 - Decision Trees
 - Bagging & Random Forest: Rooted in statistics.
 - Boosting: Utilized in both, with more attention in machine learning.



Leo Breiman,
Professor of statistics at UC Berkeley

Decision Trees : Overview

- Tree-structured model for decision-making and predictive analysis in machine learning.
- Structures decisions based on input data, making it suitable for both classification and regression tasks.
- Splits the data into subsets based on specific criteria, represented as nodes and branches in the tree in recursive manner.
- The process begins at the root node and continues until a terminal leaf node is reached, which represents the outcome or prediction.



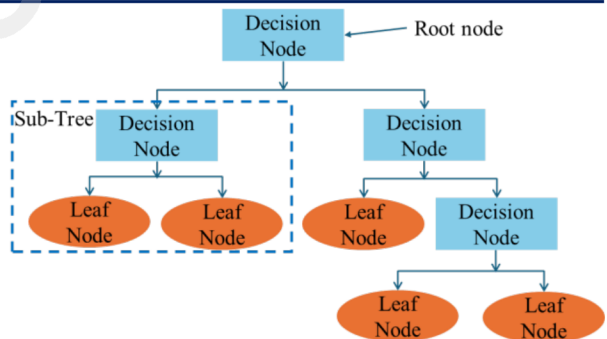
EE708

Module 4

3

Decision Trees : Terminology

- **Root Node:** The starting point representing the first decision.
- **Internal Nodes/ Decision Nodes:** splits the input based on attribute values
- **Leaf Nodes/ Terminal Nodes:** Terminal points representing the final outcomes.



- **Splitting:** Dividing a node into two or more sub-nodes based on a decision criterion.
- **Parent Node:** A node that is split into child nodes.
- **Child Node:** Nodes resulting from a split.
- **Decision Criterion:** The rule to determine how the data should be split at a decision node.
- **Branches/ Edges:** Links between nodes represent the outcomes of those decisions

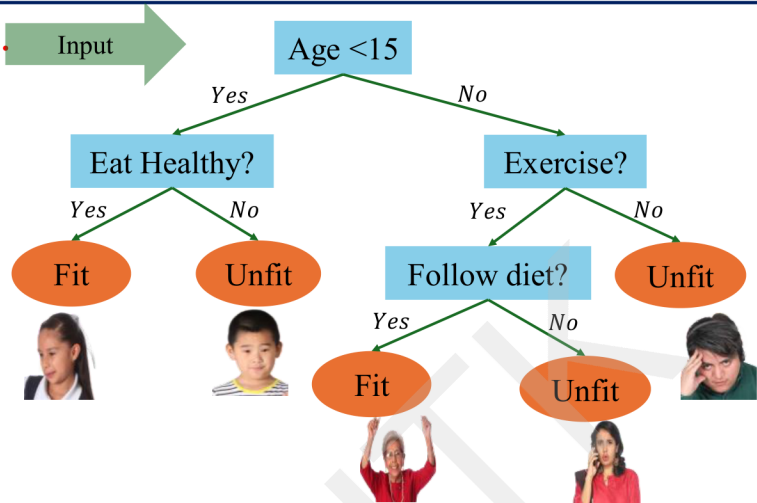
EE708

Module 4

4

Decision Trees : Identifying Fit and Unfit People

Input: Age, Gender, Activity,....



Classification and regression tree examples.

EE708

Module 4

5

How Decision Trees Work

- **Recursive Partitioning:** The dataset is recursively divided into subsets based on conditions derived from feature values.
- **Splitting Criterion:** Decisions are made using measures like:
 - *Gini Index:* Evaluates the impurity of splits (used in CART).
 - *Entropy/Information Gain:* Measures the reduction in uncertainty.
 - *Variance Reduction:* Minimize the spread of predicted values (Used in regression task).
- **Stopping Criteria:** The tree-building process stops when:
 - All data points belong to the same class.
 - The maximum depth is reached.
 - Further splitting does not improve performance.
- **How is Prediction done :** For classification, the class with the majority votes at a leaf node is assigned. For regression, the average value is computed.

EE708

Module 4

6

Classification and Regression Trees (CART)

- A classification or regression decision tree is used as a predictive model to draw conclusions about a set of observations.
- **Classification trees:** the target variable can take a discrete set of values
- Leaves represent class labels and branches represent conjunctions of features that lead to those class labels.
- **Regression trees:** the target variable can take continuous values (typically real numbers).
- **Classification and Regression Trees (CART)** is a generic term for both classification and regression trees.

EE708

Module 4

7

Learning a CART Model : LSE with Greedy Approach

- A greedy approach called recursive binary splitting is used to split the input space.
- All the values are lined up and different split points are tried and tested using a cost (loss) function.
- All input variables and the split points are evaluated in greedy manner (i.e., the very best split point is chosen each time).
- For regression, the cost function is **the sum of squared errors** across all training samples that falls in the section of interest.

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i is the actual output for a training sample, and $\hat{y}_i$ is the predicted output.

EE708

Module 4

8

Learning a CART Model : Gini Cost Function

- For classification, the Gini cost function is used to measure the purity of leaf nodes.

$$G = \sum_{k=1}^n p_k(1 - p_k)$$

where **G is the Gini cost over all classes**, and **p_k is the proportion of instances in class k** .

- The degree of the Gini index varies from 0 to 1,
 - $G = 0$: all the elements are allied to a certain class, or only one class exists there.
 - Gini index close to 1: each class contains a very small fraction of elements (high level of impurity)
 - $G = 1 - 1/n$: the elements are uniformly distributed into n equiprobable classes.

EE708

Module 4

9

Learning a CART Model

Stopping criterion

- Stopping criterion determines when to stop splitting in tree building.
- Typically, by setting a minimum number of instances required in each leaf node.
- If below this minimum, the node becomes a final leaf.

Pruning the tree

- Pruning reduces tree complexity after training to improve performance.
- It removes nodes that don't improve the model's cost function, making the tree simpler, faster, and less prone to overfitting.
- Pruning stops when no further improvements are possible.

Hyperparameters

- CART has many hyperparameters.
 - Key hyperparameter is 'max_depth'**, which controls the tree's maximum depth and ranges from 2 to 30 based on feature count.

EE708

Module 4

10

Regression Tree: Algorithm

1. Find all possible split locations, $\mathcal{S} = \left\{ \frac{x_i + x_{i+1}}{2}, \forall i \right\}$

2. For each split $s \in \mathcal{S}$:

a. Region and predicted value:

- Left split region: $R_{LHS} = \{x_i: x_i < s\}$ and $\hat{y}_{LHS} = \frac{1}{n_{LHS}} \sum_{i \in R_{LHS}} y_i$
- Right split region: $R_{RHS} = \{x_i: x_i > s\}$ and $\hat{y}_{RHS} = \frac{1}{n_{RHS}} \sum_{i \in R_{RHS}} y_i$

n_{LHS} and n_{RHS} are the number of samples in LHS and RHS region, respectively.

b. Compute Mean Squared Error (MSE):

$$MSE = \frac{n_{LHS} MSE_{LHS} + n_{RHS} MSE_{RHS}}{n_{LHS} + n_{RHS}}$$

$$MSE_{LHS} = \frac{1}{n_{LHS}} \sum_{i \in R_{LHS}} (y_i - \hat{y}_{LHS})^2, \quad MSE_{RHS} = \frac{1}{n_{RHS}} \sum_{i \in R_{RHS}} (y_i - \hat{y}_{RHS})^2$$

3. Split the tree where MSE is minimum.

4. Continue until the stopping criteria is satisfied.

EE708

Module 4

11

Regression Tree: Numerical Example

- Given 10 data samples, create the regression decision tree of maximum depth 2.

x	1	2	3	4	5	6	7	8	9	10
y	1	4	9	16	25	36	49	64	81	100

- Initial MSE (before splitting)

- Prediction for the samples:

$$\hat{y} = \bar{y} = \frac{1 + 4 + \dots + 100}{10} = 38.5$$

- Mean squared error:

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y} - y_i)^2 = \frac{1}{10} ((38.5 - 1)^2 + (38.5 - 4)^2 + \dots + (38.5 - 100)^2) = 1051.05$$

EE708

Module 4

12

Regression Tree: Numerical Example

Step 1: Possible splits: $\mathcal{S} = \{1.5, 2.5, \dots, 9.5\}$

Step 2: Calculate MSE for all possible splits

For e.g. - split at $x = 4.5$:

- Prediction for the samples:

- For $x < 4.5$:

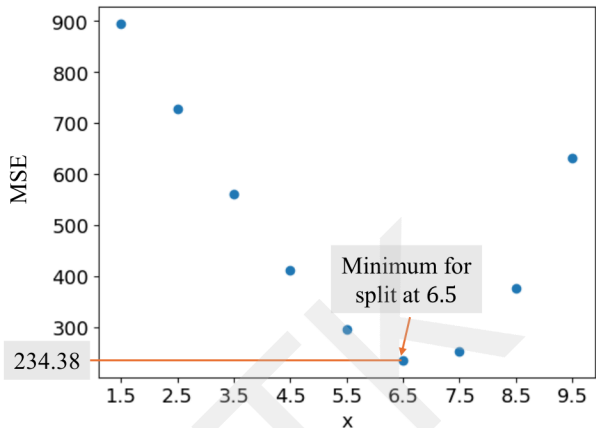
$$\hat{y}_{LHS} = \frac{1 + 4 + 9 + 16}{4} = 7.5$$

- For $x > 4.5$:

$$\hat{y}_{RHS} = \frac{25 + 36 + 49 + 64 + 81 + 100}{6} = 59.17$$

- Mean squared error:

$$MSE = \frac{n_{LHS}MSE_{LHS} + n_{RHS}MSE_{RHS}}{n} = \frac{4 \times 32.25 + 6 \times 662.47}{10} = 410.38$$



x	1	2	3	4	5	6	7	8	9	10
y	1	4	9	16	25	36	49	64	81	100

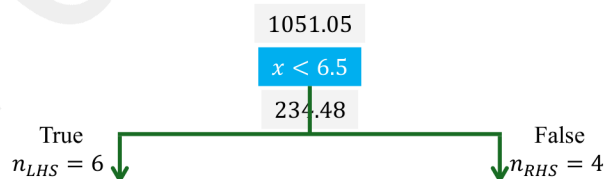
Module 4

13

Regression Tree: Numerical Example

- Updated Tree

MSE	RMSE = $\sqrt{MSE}$	δ change in RMSE
1051.05	32.42	—
234.48	15.31	17.11



EE708

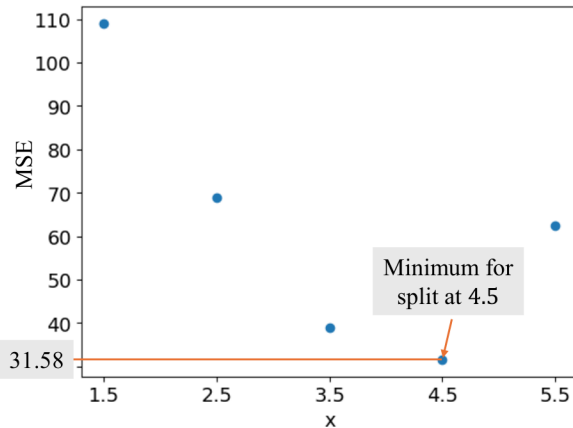
Module 4

14

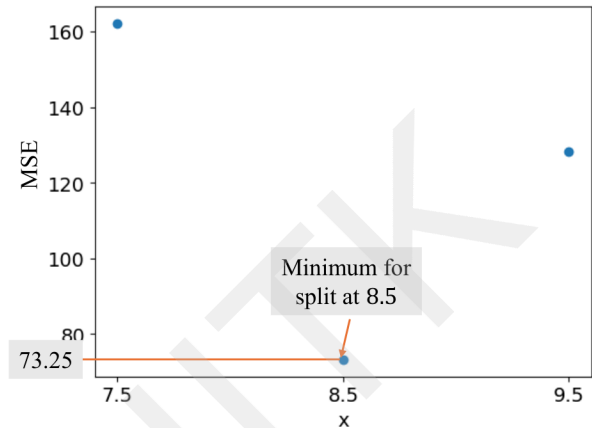
Regression Tree: Numerical Example

Step 3:

For $x < 6.5$



For $x > 6.5$



EE708

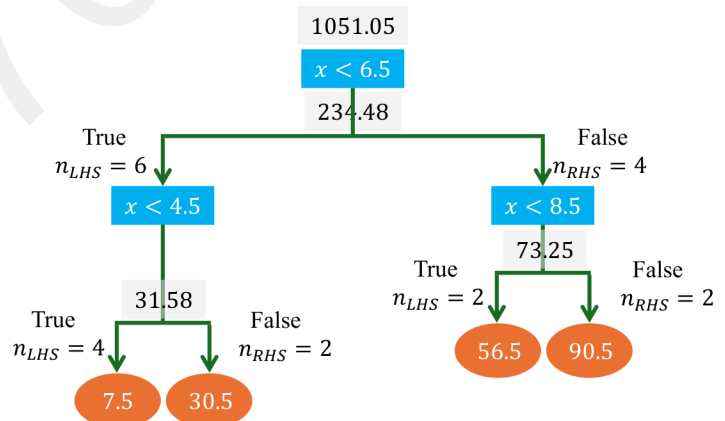
Module 4

15

Regression Tree: Numerical Example

• Updated Tree

MSE	$RMSE = \sqrt{MSE}$	δ change in RMSE
1051.05	32.42	—
234.48	15.31	17.11
73.25	8.56	8.55
31.58	5.62	2.93

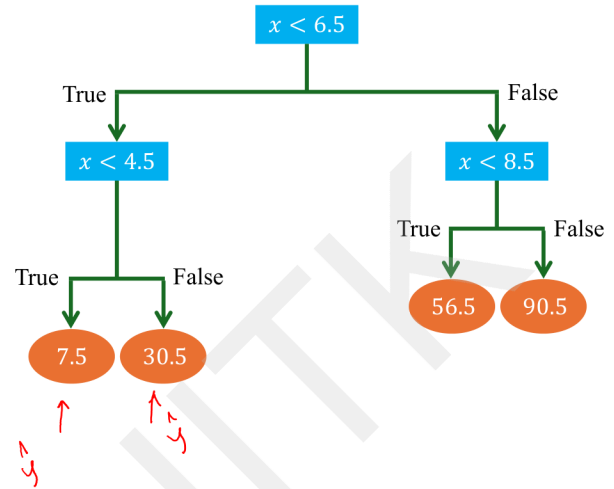
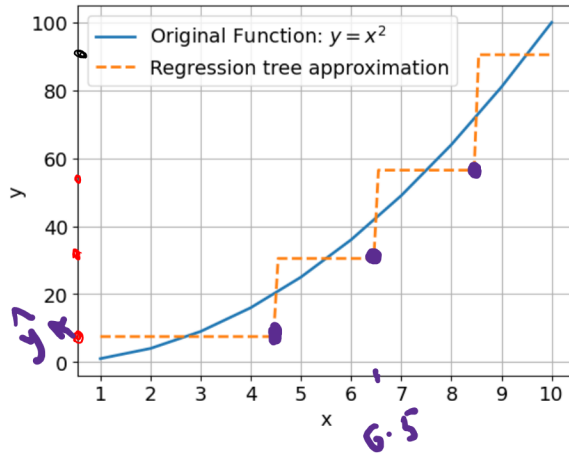


EE708

Module 4

16

Regression Tree: Numerical Example



EE708

Module 4

17

Classification Tree: Algorithm

1. Find all possible split locations for all features, $\mathcal{S}_{x_j} = \left\{ \frac{x_{j,i} + x_{j,i+1}}{2}, \forall i \in n \right\}, \forall j$
2. For each feature x_j :
 - a. For each split $s \in \mathcal{S}_{x_j}$:

a. Regions:

- Left split region: $R_{LHS} = \{x_{j,i} : x_{j,i} < s\}$
- Right split region: $R_{RHS} = \{x_{j,i} : x_{j,i} > s\}$

b. Compute Gini impurity:

$$G_{split} = \frac{n_{LHS}G_{LHS} + n_{RHS}G_{RHS}}{n_{LHS} + n_{RHS}}$$

$$G_{LHS} = 1 - \sum_{j=1}^C \left(\frac{n_{c,LHS}}{n_{LHS}} \right)^2, \quad G_{RHS} = 1 - \sum_{j=1}^C \left(\frac{n_{c,RHS}}{n_{RHS}} \right)^2$$

n_{LHS} and n_{RHS} are the number of samples in LHS and RHS region, respectively.

$n_{c,LHS}$ is the number of samples in R_{LHS} that belongs to class c .

3. Split the tree where Gini impurity is minimum.
4. Continue until the stopping criteria is satisfied.

EE708

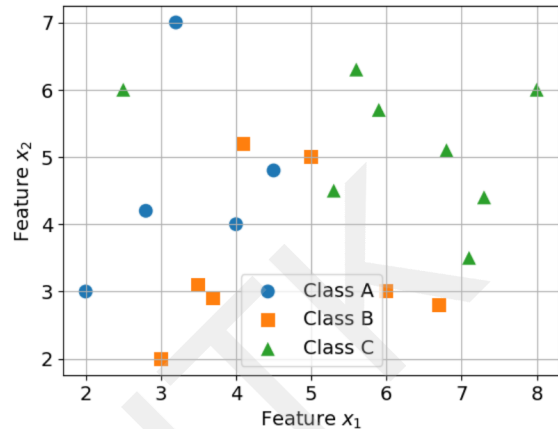
Module 4

18

Classification Tree: Numerical Example

- Given 20 data samples, create the classification decision tree.

x_1	x_2	y	x_1	x_2	y
2	3	A	7.1	3.5	C
3	2	B	2.8	4.2	A
4	4	A	3.5	3.1	B
5	5	B	5.6	6.3	C
6	3	B	6.8	5.1	C
2.5	6	C	3.7	2.9	B
3.2	7	A	4.5	4.8	A
4.1	5.2	B	5.9	5.7	C
5.3	4.5	C	7.3	4.4	C
6.7	2.8	B	8	6	C



EE708

Module 4

19

Classification Tree: Numerical Example

- Gini impurity for root node

$$G = 1 - \left(\frac{5}{20}\right)^2 - \left(\frac{7}{20}\right)^2 - \left(\frac{8}{20}\right)^2 = 0.655$$

- Step 1: Possible splits

$$\mathcal{S}_{x_1} = \{2.25, 2.65, 2.90, \dots, 7.65\}, \quad \mathcal{S}_{x_2} = \{2.40, 2.85, 3, \dots, 6.65\}$$

Note : Splits must be calculated after arranging x_1 and x_2 in ascending order, from the original data

EE708

Module 4

20

Classification Tree: Numerical Example

- Step 2: Compute Gini impurity for all splits

For e.g. - split at $x_1 = 4.75$:

For $x_1 < 4.75$

$$G_{LHS} = 1 - \left(\frac{5}{10}\right)^2 - \left(\frac{4}{10}\right)^2 - \left(\frac{1}{10}\right)^2 = 0.58$$

x_1	x_2	y
2	3	A
3	2	B
4	4	A
2.5	6	C
3.2	7	A
4.1	5.2	B
2.8	4.2	A
3.5	3.1	B
3.7	2.9	B
4.5	4.8	A

$n_{A,LHS}$	$n_{B,LHS}$	$n_{C,LHS}$
5	4	1

EE708

Module 4

21

Classification Tree: Numerical Example

For $x_1 > 4.75$

$$G_{RHS} = 1 - \left(\frac{0}{10}\right)^2 - \left(\frac{3}{10}\right)^2 - \left(\frac{7}{10}\right)^2 = 0.42$$

$$G_{split} = \frac{n_{LHS}G_{LHS} + n_{RHS}G_{RHS}}{n_{LHS} + n_{RHS}} = \frac{10 \times 0.58 + 10 \times 0.42}{10 + 10} = 0.5$$

x_1	x_2	y
5	5	B
6	3	B
5.3	4.5	C
6.7	2.8	B
7.1	3.5	C
5.6	6.3	C
6.8	5.1	C
5.9	5.7	C
7.3	4.4	C
8	6	C

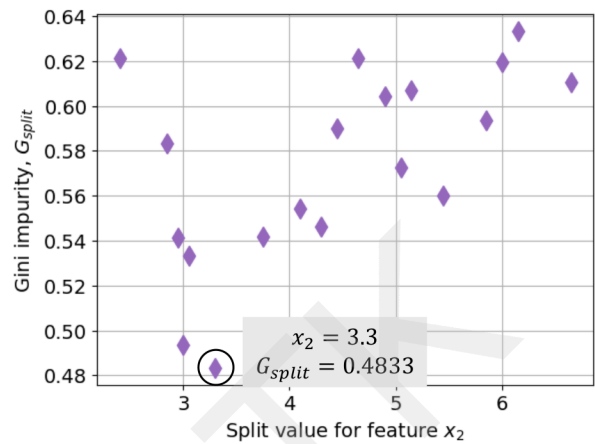
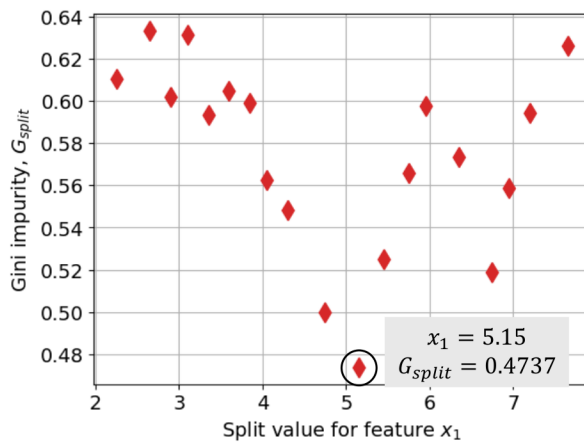
$n_{A,LHS}$	$n_{B,LHS}$	$n_{C,LHS}$
0	3	7

EE708

Module 4

22

Classification Tree: Numerical Example



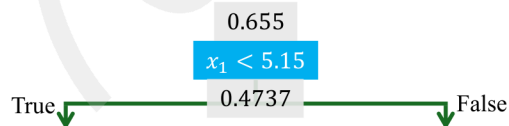
- Best split with x_1 at 5.15.

EE708

Module 4

23

Classification Tree: Numerical Example

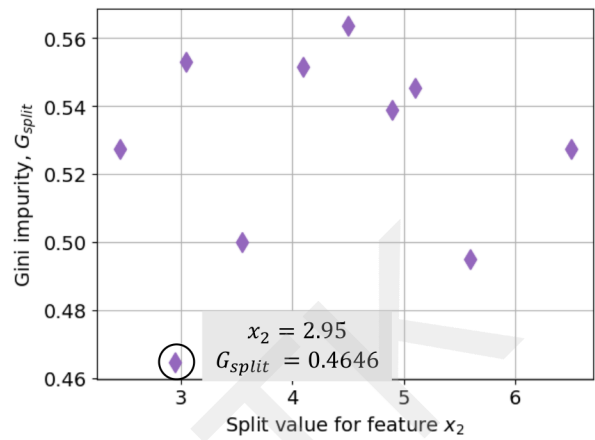
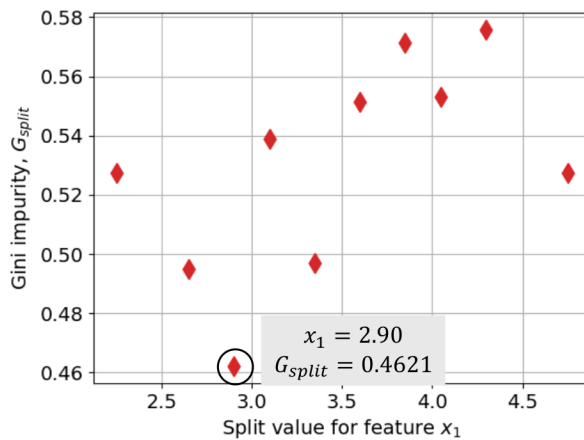


EE708

Module 4

24

Classification Tree: Numerical Example



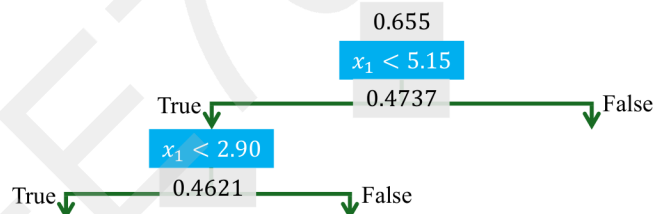
- Best split with x_1 at 2.90.

EE708

Module 4

25

Classification Tree: Numerical Example

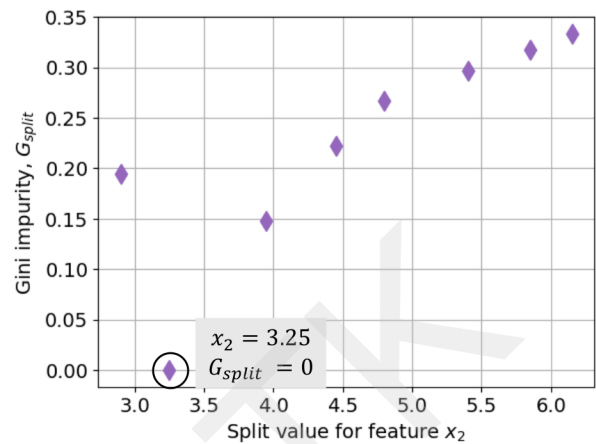
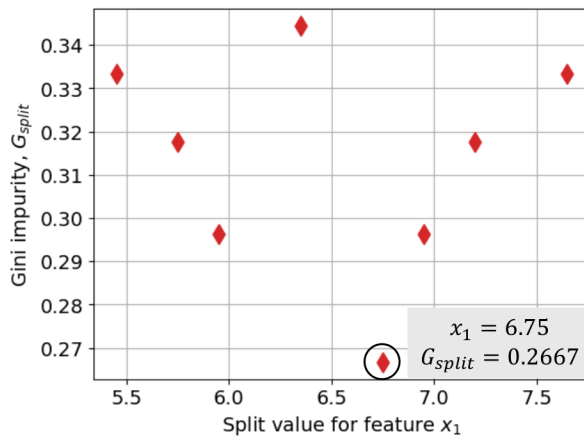


EE708

Module 4

26

Classification Tree: Numerical Example



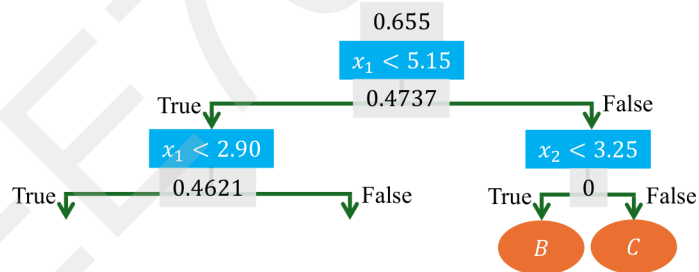
- Best split with x_2 at 3.25.

EE708

Module 4

27

Classification Tree: Numerical Example

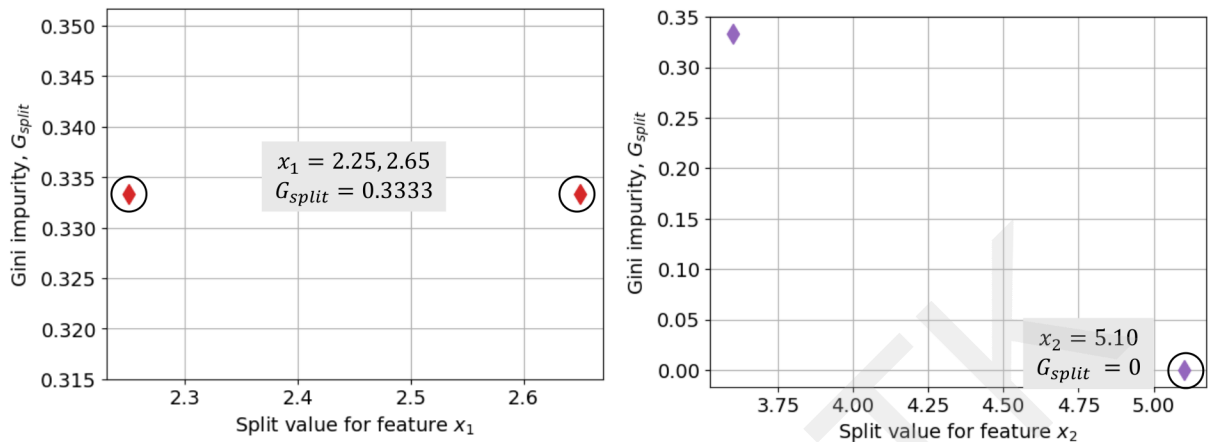


EE708

Module 4

28

Classification Tree: Numerical Example



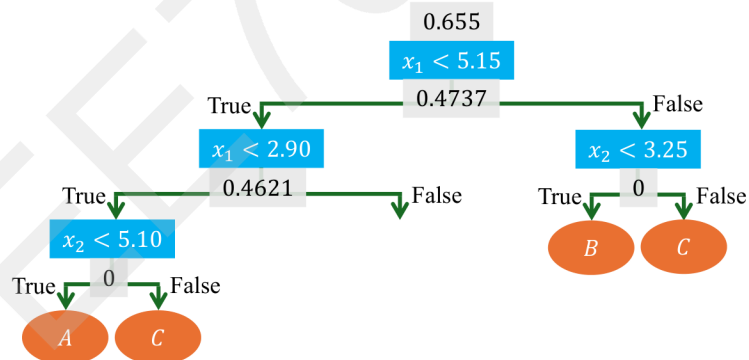
- Best split with x_2 at 5.10.

EE708

Module 4

29

Classification Tree: Numerical Example

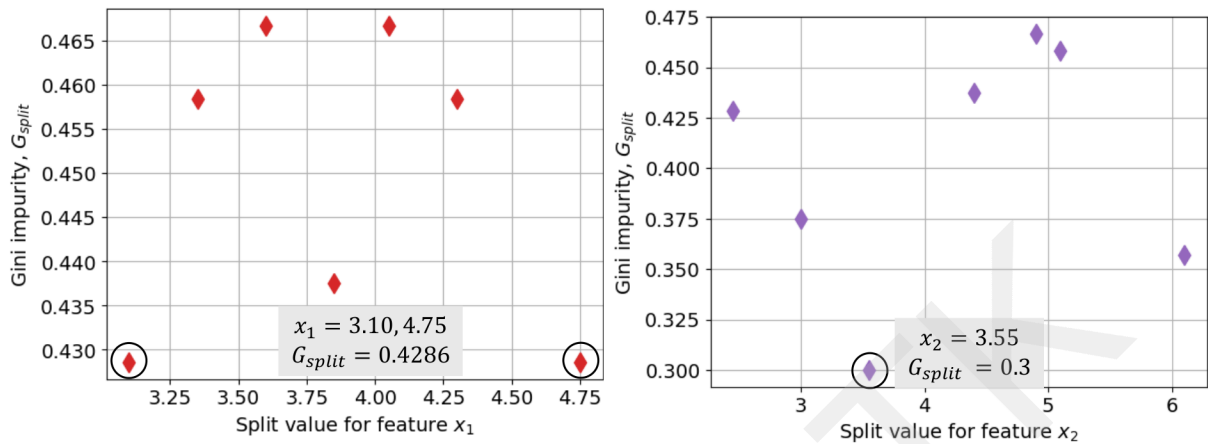


EE708

Module 4

30

Classification Tree: Numerical Example



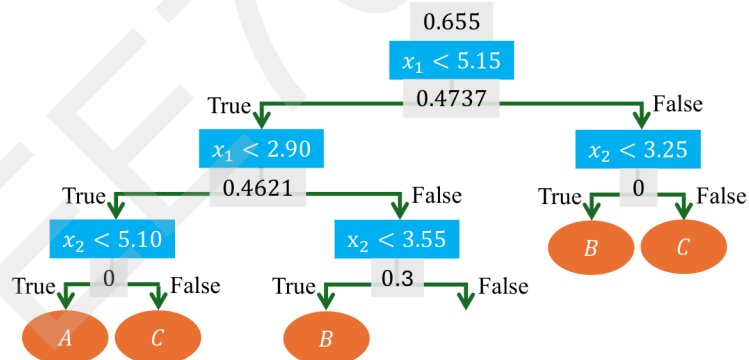
- Best split with x_2 at 3.55

EE708

Module 4

31

Classification Tree: Numerical Example

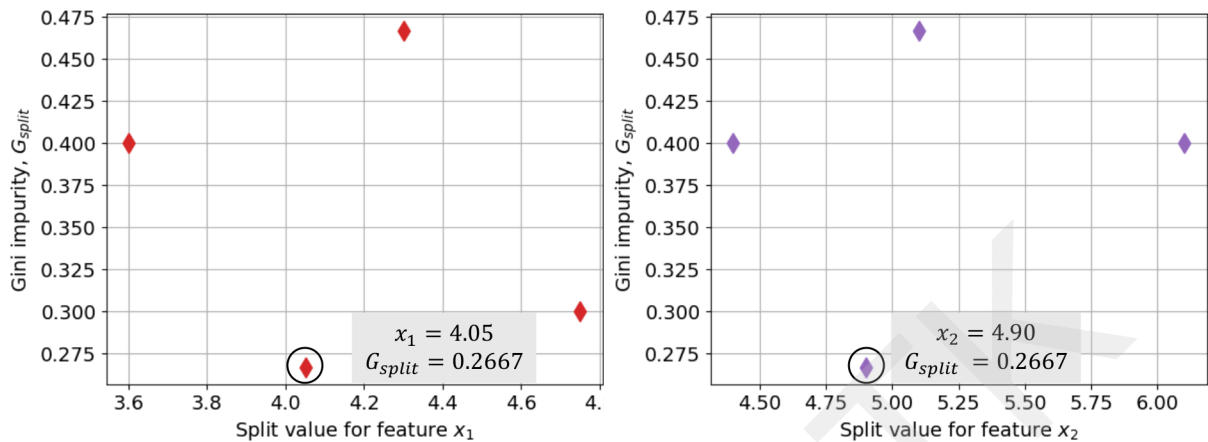


EE708

Module 4

32

Classification Tree: Numerical Example



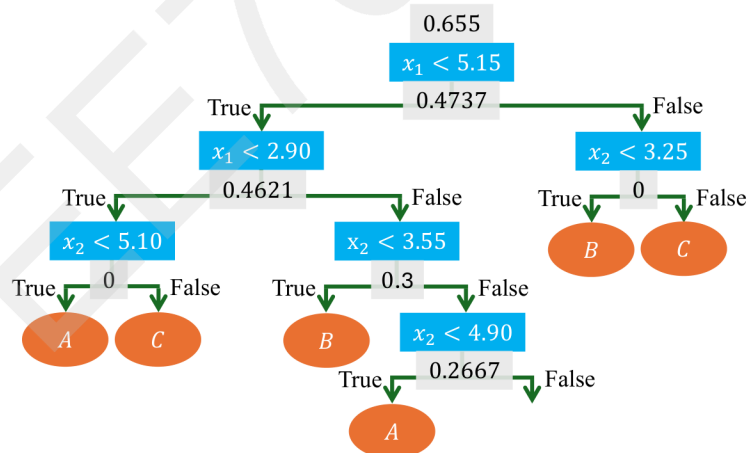
- Best split with x_2 at 4.90.

EE708

Module 4

33

Classification Tree: Numerical Example

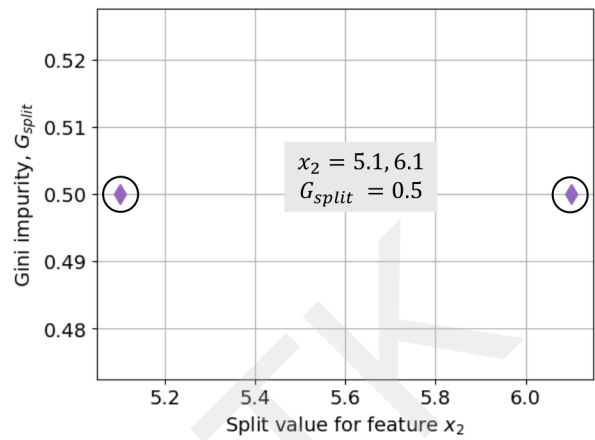
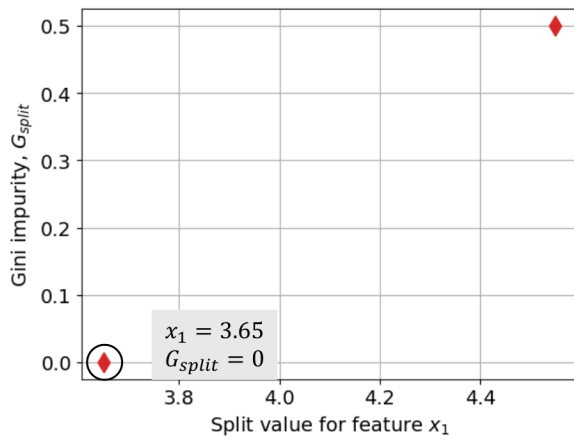


EE708

Module 4

34

Classification Tree: Numerical Example



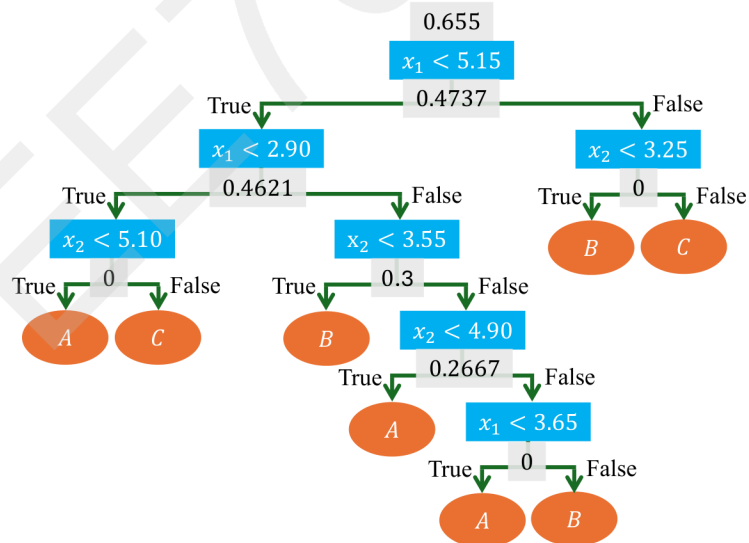
- Best split with x_1 at 3.65.

EE708

Module 4

35

Classification Tree: Numerical Example

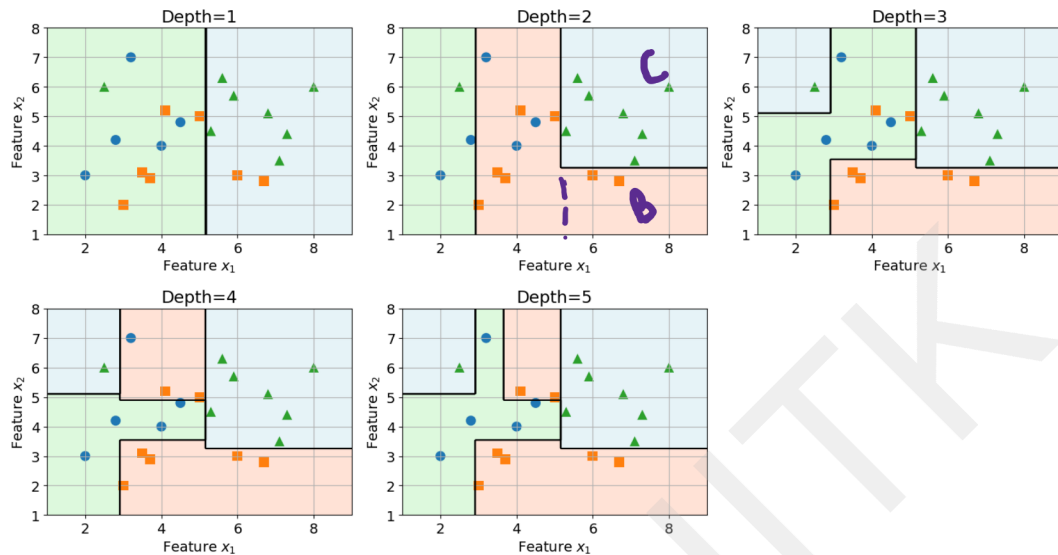


EE708

Module 4

36

Classification Tree: Numerical Example

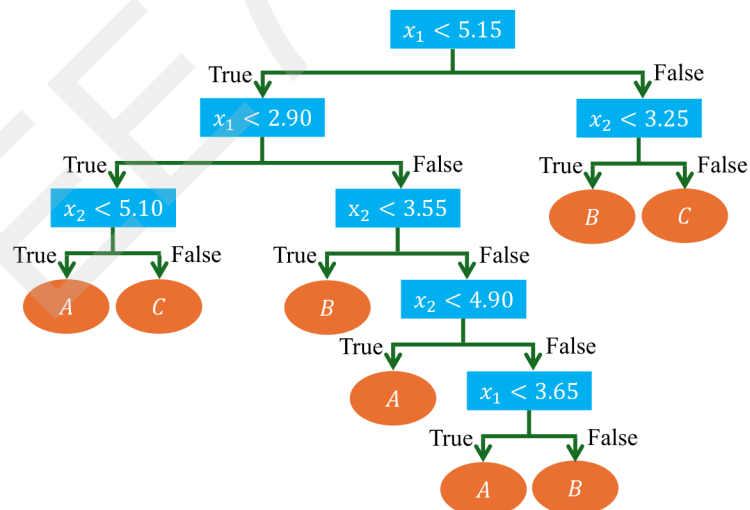


EE708

Module 4

37

Classification Tree: Numerical Example

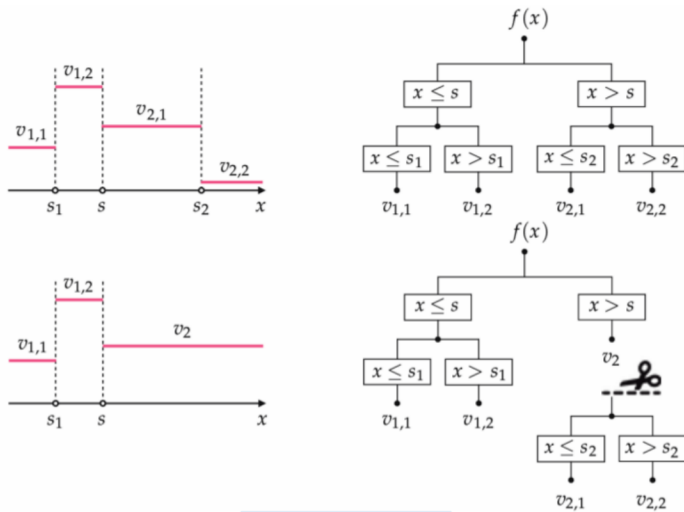


EE708

Module 4

38

Pruning



A fully grown tree of depth two with four leaves.

A pruned version of the original tree wherein the leaves $v_{2,1}$ and $v_{2,2}$ are pruned and replaced by a single leaf.

Pruning illustrated

EE708

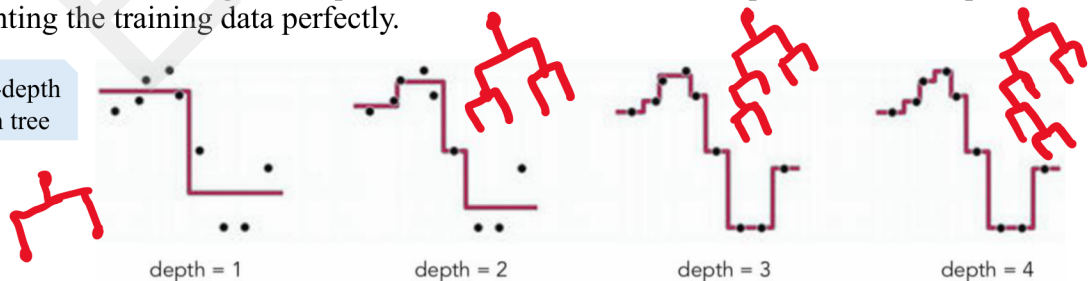
Module 4

39

Maximum-depth Tree

- Growing a maximum-depth regression tree.
- The recursive procedure for growing a deep regression tree is illustrated in figure.
- Begin (on the left) by fitting a stump (tree with single decision node) to the original dataset.
- As we move from left to right the recursion proceeds, with each leaf of the preceding tree split in order to create the next, deeper tree.
- As can be seen in the rightmost panel, a tree with maximum depth of four is capable of representing the training data perfectly.

Maximum-depth regression tree

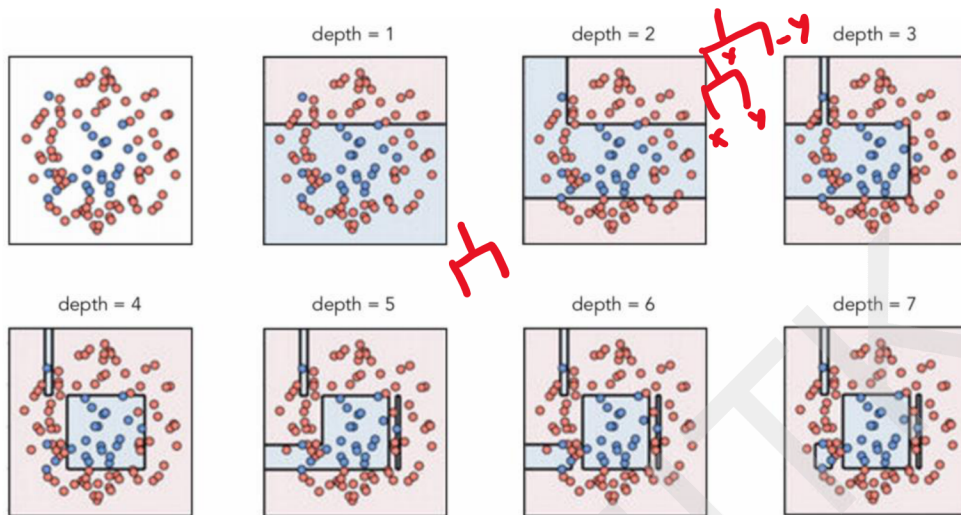


EE708

Module 4

40

Maximum-depth Tree



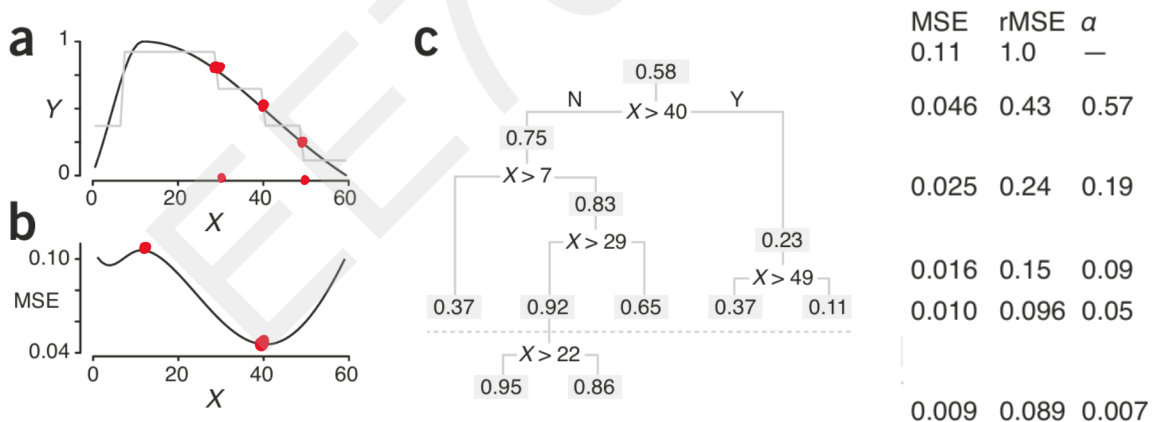
Maximum-depth classification tree

EE708

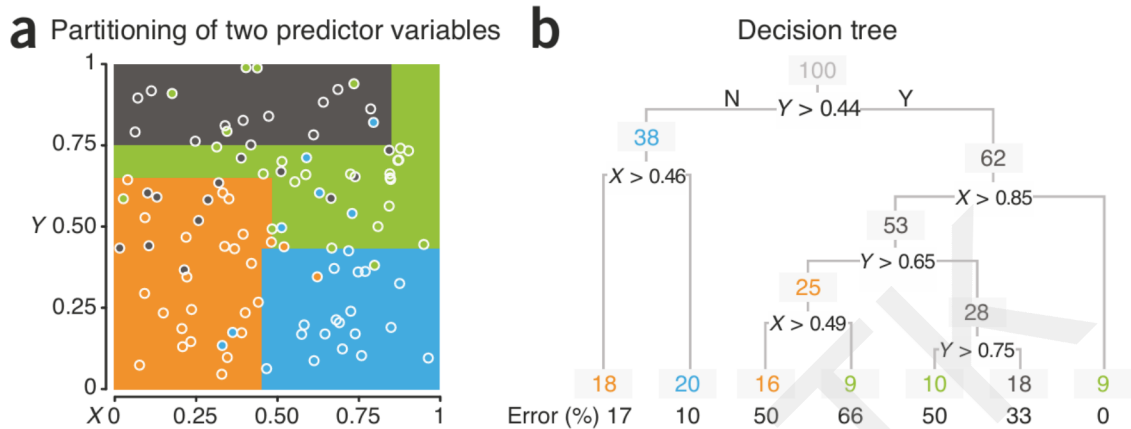
Module 4

41

Classification and Regression Trees (CART) : Example 1



Classification and Regression Trees (CART) : Example 2



Classification trees (a) Classification boundaries for an $n = 100$ sample with two predictor variables (X , Y) and four categories (colors). (b) Decision tree built with $\alpha = 0.01$ of the data set in a.

EE708

Module 4

[Source: Classification and regression trees | Nature Methods](#)

43

Ensemble Models : Bagging and Boosting

- Combine multiple classifiers into a single meta-classifier.
- It offers better accuracy and robustness than individual models.
- Two popular ensemble method: (i) **Bagging**, (ii) **Boosting**
- **Bagging**: parallel ensemble technique, trains multiple models on random subsets of data to reduce variance and improve stability.
- **Boosting**: sequential ensemble approach, builds models iteratively, with each model correcting the errors of the previous one to reduce bias.
- Ensemble methods are flexible (low bias) and less data sensitive (low variance), achieving better generalization.
- Bagging and boosting can be used for classification and regression problems.

EE708

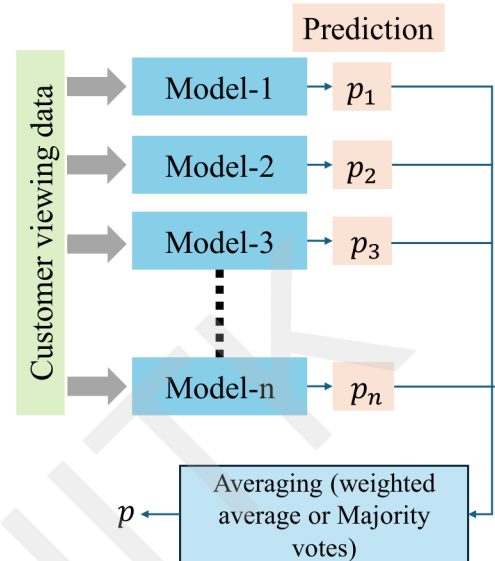
Module 4

45

Ensemble Models: With an Example

- The simple version of ensembles is as follows:
 - Develop a predictive model and record the predictions for a given data set.
 - Repeat for multiple models on the same data.
 - For each record to be predicted, take an average (or a weighted average, or a majority vote) of the predictions.

Example : Predicting the rating that a Netflix customer would award a movie.



EE708

Module 4

46

Bagging

- Also called **bootstrap aggregation**, introduced by Leo Breiman in 1994.
- Let Y be the response and $\mathbf{X} = X_1, X_2, \dots, X_P$ with N records representing P predictor variables.
- Algorithm:
 - Initialize:** M (no. of models), n (no. of records to choose ($n < N$)), counter $m = 1$.
 - Bootstrap (with replacement),** n records to form a subsample Y_m and $\mathbf{X}_m$ (the **bag**).
 - Train a model using Y_m and $\mathbf{X}_m$ to create a set of decision rules $\hat{f}_m(\mathbf{X})$
 - Increment the model counter: $m = m + 1$. If $m \leq M$, repeat **Step 2**.

The bagged estimate for $\mathbf{X}$ is given by:

$$\hat{f} = \frac{1}{M} \sum_{m=1}^M \hat{f}_m(\mathbf{X})$$

EE708

Module 4

47

Random Forest

- Bagged estimate based on decision tree models also called **Bagged decision trees**.
1. Take a bootstrap (with replacement) subsample from the records.
 2. First split, sample p predictor variables at random without replacement, where $p < P$.
 3. For each variables $X_{j(1)}, X_{j(2)}, \dots, X_{j(p)}$ apply the splitting algorithm:
 - a. For each value $s_{j(k)}$ of $X_{j(k)}$:
 - i. Split the records in partition A : if $X_{j(k)} < s_{j(k)}$; partition B : $X_{j(k)} \geq s_{j(k)}$
 - ii. Measure the homogeneity of classes within each subpartition of A .
 - b. Select the value $s_{j(k)}$ that produces maximum within-partition homogeneity of class.
 4. Select the variable $X_{j(k)}$ and split value $s_{j(k)}$ that maximizes the homogeneity of class.
 5. Proceed to the next split and repeat the previous steps, starting with Step 2.
 6. Continue with additional splits, following the same procedure until the tree is grown.
 7. Go back to Step 1, take another bootstrap subsample, and start the process over again.

EE708

Module 4

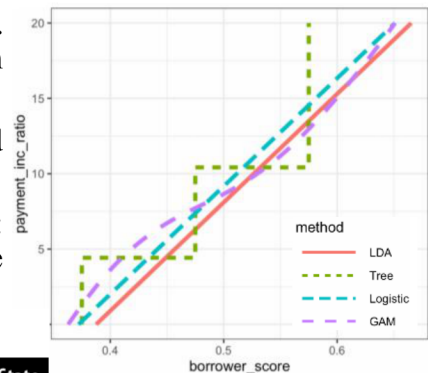
48

Case Study: Loan Default Data

- A few records of personal loan data from *LendingClub*. *LendingClub* is a leader in peer-to-peer lending in which pools of investors make personal loans to individuals.
- Goal: predict the outcome of a new potential loan: paid off versus default.
- From the loan data just using just two predictor variables: `borrower_score` and `payment_inc_ratio` (needs to be calculated) prediction can be done.

A few records and columns for LendingClub loan data

Outcome	Loan amount	Income	Purpose	Years employed	Home ownership	State
Paid off	10000	79100	debt_consolidation	11	MORTGAGE	NV
Paid off	9600	48000	moving	5	MORTGAGE	TN
Paid off	18800	120036	debt_consolidation	11	MORTGAGE	MD
Default	15250	232000	small_business	9	MORTGAGE	CA
Paid off	17050	35000	debt_consolidation	4	RENT	MD
Paid off	5500	43000	debt_consolidation	4	RENT	KS



Comparison of the classification rules for four different methods

49

Case Study: Loan Default Data

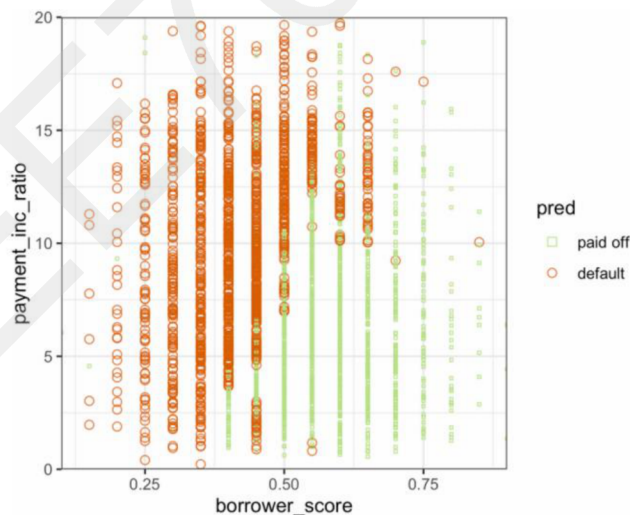
- Random forest identifies likely defaulters based on borrower scores and payment-to-income ratios.
- Random Forest Parameters:
 - By default, 500 trees are trained in a random forest.
 - Variables are randomly selected for splitting at each stage,
- Out-of-Bag (OOB) Error:
 - OOB error is the error rate calculated on data excluded from each bootstrap sample.
 - The OOB error can be plotted against the number of trees to evaluate model performance.

EE708

Module 4

50

Case Study: Loan Default Data



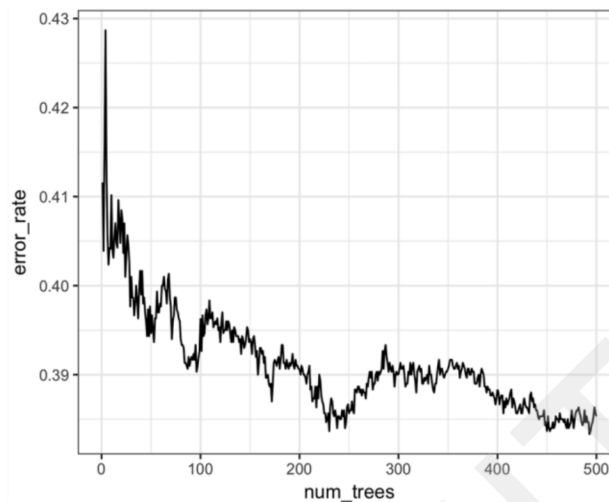
The predicted outcomes from the random forest applied to the loan default data

EE708

Module 4

51

Case Study: Loan Default Data



Improvement in accuracy of the random forest with the addition of more trees

EE708

Module 4

52

Adaptive Boosting (AdaBoost)

- Adaptive Boosting or AdaBoost is a sequential boosting technique where each model aims to correct the errors of its predecessor by modifying the sample distribution.
- In each iteration, weights are increased for wrongly predicted instances and decreased for correctly predicted ones, focusing more on harder cases.
- Steps of the AdaBoost algorithm include:
 1. Initially, all observations are assigned equal weights.
 2. A model is built on a data subset, and errors are calculated by comparing predictions with actual values.
 3. The next model is trained with higher weights assigned to misclassified instances, based on error values.
 4. This process is repeated until errors stabilize or a maximum number of iterations is reached.

EE708

Module 4

54

Adaptive Boosting (AdaBoost)

- AdaBoost's key hyperparameters are:
 - Learning rate: Controls the contribution of each classifier, with typical values being 0.001, 0.01, and 0.1.
 - Number of estimators: Represents the number of trees, usually increased from 10 to 1,000 until no further improvement is observed.
- Advantages:
 - High precision and ability to model nonlinear relationships. Requires less parameter tuning compared to other models.
 - Does not need scaled data and works effectively with minimal adjustments.
- Disadvantages:
 - Training can be time-consuming.
 - Sensitive to noisy data and outliers.
 - Performance may degrade with data imbalance, reducing classification accuracy.

EE708

Module 4

55

Boosting Algorithm

- The easiest to understand is [AdaBoost](#), which proceeds as follows:
 1. Initialize:
 - M (maximum number of models to be fit) and set the iteration counter $m = 1$.
 - **Observation weights** $w_i = \frac{1}{N}$ for $i = 1, 2, \dots, N$.
 - Ensemble model $\hat{F}_0 = 0$.
 2. Using $w_1, w_2, \dots, w_N$, train a model $\hat{f}_m$ that minimizes the weighted error e_m .
 3. Add the model to the ensemble: $\hat{F}_m = \hat{F}_{m-1} + \alpha_m * \hat{f}_m$ where $\alpha_m = \frac{1}{2} \ln \left(\frac{1-e_m}{e_m} \right)$
 4. Update the weights $w_1, w_2, \dots, w_N$ using α_m

$$w_i^{\text{updated}} = \frac{1}{\tilde{w}} \times \begin{cases} w_i \exp(-e_m), & \hat{y}_i = y_i \\ w_i \exp(e_m), & \text{otherwise} \end{cases}$$

$\tilde{w}$ is the normalization factor for updated weights to sum 1
 5. Increment the model counter $m = m + 1$. If $m \leq M$, go to step 2.
 6. The boosted estimate is: $\hat{F} = \sum_{i=1}^M \alpha_i * \hat{f}_i$

EE708

Module 4

56

XGBoost

- XGBoost is a popular implementation of stochastic gradient boosting
- Key Parameters:
 - *subsample*: Controls the fraction of observations sampled at each iteration. Sampling is done without replacement, introducing randomness similar to random forests.
 - *eta*: A shrinkage parameter applied to weights α_m in the boosting algorithm. Smaller values reduce the magnitude of updates, helping to prevent overfitting by making the model less sensitive to each individual iteration.
- Benefits:
 - Subsampling helps to boost mimic random forests while maintaining iterative focus on reducing errors.
 - Shrinkage (via eta) reduces the risk of overfitting, as smaller weight adjustments create a more robust and generalized model.

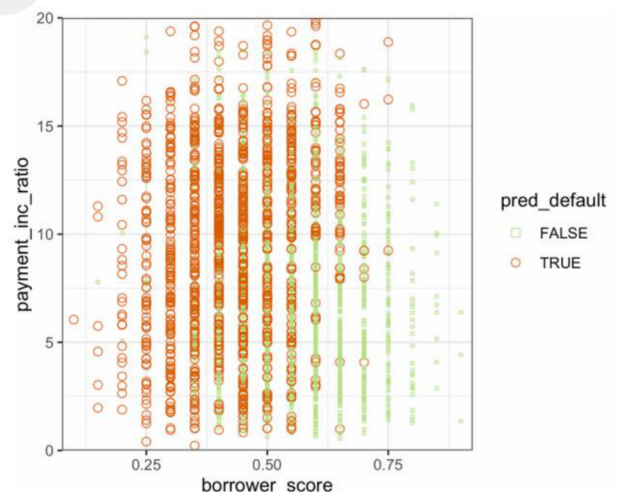
EE708

Module 4

58

XGBoost

- Predictions align closely with the random forest model.
- High borrower scores still show some defaults, indicating model sensitivity to noisy or outlier data.
- Captures complex patterns but requires careful tuning to handle edge cases effectively.



The predicted outcomes from XGBoost applied to the loan default data

EE708

Module 4

59

Gradient Boosting Method

- Sequential boosting technique that corrects errors from previous models by adding their underfitted predictions to the ensemble.
- The steps of the gradient boosting algorithm are:
 1. A weak learner (first model) is built on a subset of data, and predictions are made for the whole dataset.
 2. Errors are calculated by comparing predictions with actual values, using a loss function.
 3. A new model is trained using the previous errors as the target, aiming to minimize these errors by finding the best split. Predictions from the new model are combined with the previous ones, and new errors are calculated.
 4. This process repeats until the error stabilizes or the maximum number of estimators is reached.
- Unlike AdaBoost, which adjusts instance weights, gradient boosting fits new predictors to the residual errors from previous models.

EE708

Module 4

60

Variable Importance in Ensemble Methods

- Ensemble Methods (eg. Random forests) identify important predictors and reveal complex interactions between variables.
- **Mean Decrease in Accuracy:** Measures the reduction in model accuracy when a variable's values are randomly permuted.
- This is calculated using out-of-bag (OOB) data, providing a cross-validated estimate of importance.
- **Mean Decrease in Node Impurity:** Evaluates how much a variable contributes to improving node purity (e.g., Gini impurity reduction) across all splits in the forest.
- This measure is based on training data and is less reliable than the OOB-based method.

EE708

Module 4

62

Variable Importance

How the Measures Work:

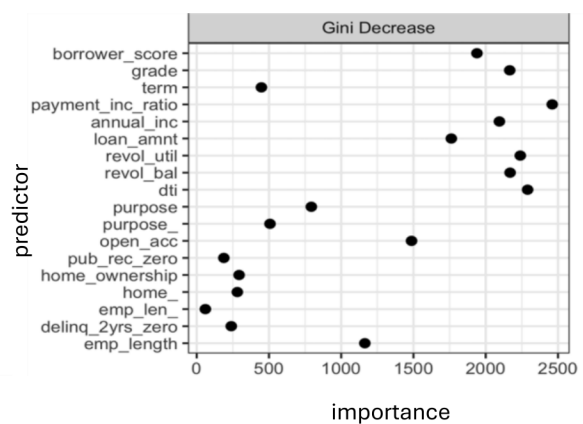
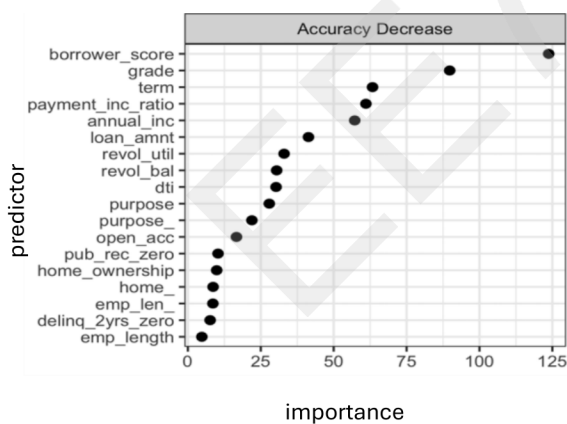
- Decrease in Accuracy:
 - Randomly permuting a variable removes its predictive power.
 - The drop in accuracy is a strong indicator of the variable's importance.
- Decrease in Node Impurity:
 - Tracks the Gini impurity reduction for nodes where the variable is used.
 - Measures the variable's role in improving class homogeneity during tree construction.

EE708

Module 4

63

Variable Importance



The importance of variables for the full model fit to the loan data

EE708

Module 4

64